Mental Health Dataset

Data Cleaning/Normalization.

I will perform quality control by reviewing the dataset and addressing the following errors commonly found in data files.

Removal of duplicates

I decided to run a duplicate-check across all columns. Person_ID is the column that should be unique. The rest of the columns were numerical in nature or had commonly shared string values, such as Occupation, that one would expect to have recurring values. Given that, I used the .drop_duplicates function targeting the Person_ID column, keeping the first occurrence and dropping any subsequent occurrences. In my output section you will see that there are 0 duplicates in the Person_ID column

```
df = df.drop_duplicates(subset=['Person_ID'], keep='first')
```

Code:

```
#duplicates:
print("\nDuplicate check for all individual columns: ")
print(" ")
#Storing duplicates for each column on a new variable
Person_ID_duplicates = df[df.duplicated(subset=['Person_ID'])]['Person_ID'].count()
Age_duplicates = df[df.duplicated(subset=['Age'])]['Age'].count()
Gender_duplicates = df[df.duplicated(subset=['Gender'])]['Gender'].count()
Occupation_duplicates = df[df.duplicated(subset=['Occupation'])]['Occupation'].count()
Daily_Screen_Time_duplicates = df[df.duplicated(subset=['Daily_Screen_Time'])]['Daily_Screen_Time'].count()
Social_Media_Usage_duplicates = df[df.duplicated(subset=['Social_Media_Usage'])]['Social_Media_Usage'].count()
Night_Usage_duplicates = df[df.duplicated(subset=['Night_Usage'])]['Night_Usage'].count()
Sleep_Hours_duplicates = df[df.duplicated(subset=['Sleep_Hours'])]['Sleep_Hours'].count()
Stress_Level_duplicates = df[df.duplicated(subset=['Stress_Level'])]['Stress_Level'].count()
Work_Study_Hours_duplicates = df[df.duplicated(subset=['Work_Study_Hours'])]['Work_Study_Hours'].count()
Physical_Activity_duplicates = df[df.duplicated(subset=['Physical_Activity'])]['Physical_Activity'].count()
Social_Interaction_Score_duplicates = df[df.duplicated(subset=['Social_Interaction_Score'])]['Social_Interaction_Score'].count()
Caffeine_Intake_duplicates = df[df.duplicated(subset=['Caffeine_Intake'])]['Caffeine_Intake'].count()
Smoking_duplicates = df[df.duplicated(subset=['Smoking'])]['Smoking'].count()
Alcohol_duplicates = df[df.duplicated(subset=['Alcohol'])]['Alcohol'].count()
Depression_duplicates = df[df.duplicated(subset=['Depression'])]['Depression'].count()
Anxiety_duplicates = df[df.duplicated(subset=['Anxiety'])]['Anxiety'].count()
Burnout_duplicates = df[df.duplicated(subset=['Burnout'])]['Burnout'].count()
```

```python
df = df.drop_duplicates(subset=['Person_ID'], keep='first')

print("Person_ID duplicates: ", Person_ID_duplicates)
print("Age duplicates: ", Age_duplicates)
print("Gender duplicates: ", Gender_duplicates)
print("Occupation duplicates: ", Occupation_duplicates)
print("Daily_Screen_Time duplicates: ", Daily_Screen_Time_duplicates)
print("Social_Media_Usage duplicates: ", Social_Media_Usage_duplicates)
print("Night_Usage duplicates: ", Night_Usage_duplicates)
print("Sleep_Hours duplicates: ", Sleep_Hours_duplicates)
print("Stress_Level duplicates: ", Stress_Level_duplicates)
print("Work_Study_Hours duplicates: ", Work_Study_Hours_duplicates)
print("Physical_Activity duplicates: ", Physical_Activity_duplicates)
print("Social_Interaction_Score duplicates: ", Social_Interaction_Score_duplicates)
print("Caffeine_Intake duplicates: ", Caffeine_Intake_duplicates)
print("Smoking duplicates: ", Smoking_duplicates)
print("Alcohol duplicates: ", Alcohol_duplicates)
print("Depression duplicates: ", Depression_duplicates)
print("Anxiety duplicates: ", Anxiety_duplicates)
print("Burnout duplicates: ", Burnout_duplicates)
```

Output:

```
Duplicate check for all individual columns:

Person_ID duplicates:  0
Age duplicates:  1956
Gender duplicates:  1998
Occupation duplicates:  1997
Daily_Screen_Time duplicates:  1899
Social_Media_Usage duplicates:  1929
Night_Usage duplicates:  1998
Sleep_Hours duplicates:  1939
Stress_Level duplicates:  1990
Work_Study_Hours duplicates:  1899
Physical_Activity duplicates:  1997
Social_Interaction_Score duplicates:  1990
Caffeine_Intake duplicates:  1995
Smoking duplicates:  1998
Alcohol duplicates:  1998
Depression duplicates:  1998
Anxiety duplicates:  1998
Burnout duplicates:  1998
```

Missing Value Check

For missing values, I used Python's .isnull() and .count() to count missing values in each column. There were no missing values. **The dataset is full.**

Code:

```
#Checking for missing values:

Person_ID_Missing_values = df['Person_ID'].isnull().sum()
Age_Missing_values = df['Age'].isnull().sum()
Gender_Missing_values = df['Gender'].isnull().sum()
Occupation_Missing_values = df['Occupation'].isnull().sum()
Daily_Screen_Time_Missing_values = df['Daily_Screen_Time'].isnull().sum()
Social_Media_Usage_Missing_values = df['Social_Media_Usage'].isnull().sum()
Night_Usage_Missing_values = df['Night_Usage'].isnull().sum()
Sleep_Hours_Missing_values = df['Sleep_Hours'].isnull().sum()
Stress_Level_Missing_values = df['Stress_Level'].isnull().sum()
Work_Study_Hours_Missing_values = df['Work_Study_Hours'].isnull().sum()
Physical_Activity_Missing_values = df['Physical_Activity'].isnull().sum()
Social_Interaction_Score_Missing_values = df['Social_Interaction_Score'].isnull().su
Caffeine_Intake_Missing_values = df['Caffeine_Intake'].isnull().sum()
Smoking_Missing_values = df['Smoking'].isnull().sum()
Alcohol_Missing_values = df['Alcohol'].isnull().sum()
Depression_Missing_values = df['Depression'].isnull().sum()
Anxiety_Missing_values = df['Anxiety'].isnull().sum()
Burnout_Missing_values = df['Burnout'].isnull().sum()
print("_" * 100)
print(" ")
print("Missing Value Check: ")
print(" ")
```

```python
print("Person_ID Missing values: ", Person_ID_Missing_values)
print("Age Missing values: ", Age_Missing_values)
print("Gender Missing values: ", Gender_Missing_values)
print("Occupation Missing values: ", Occupation_Missing_values)
print("Daily_Screen_Time Missing values: ", Daily_Screen_Time_Missing_values)
print("Social_Media_Usage Missing values: ", Social_Media_Usage_Missing_values)
print("Night_Usage Missing values: ", Night_Usage_Missing_values)
print("Sleep_Hours Missing values: ", Sleep_Hours_Missing_values)
print("Stress_Level Missing values: ", Stress_Level_Missing_values)
print("Work_Study_Hours Missing values: ", Work_Study_Hours_Missing_values)
print("Physical_Activity Missing values: ", Physical_Activity_Missing_values)
print("Social_Interaction_Score Missing values: ", Social_Interaction_Score_Missing_values)
print("Caffeine_Intake Missing values: ", Caffeine_Intake_Missing_values)
print("Smoking Missing values: ", Smoking_Missing_values)
print("Alcohol Missing values: ", Alcohol_Missing_values)
print("Depression Missing values: ", Depression_Missing_values)
print("Anxiety Missing values: ", Anxiety_Missing_values)
print("Burnout Missing values: ", Burnout_Missing_values)
print("_" * 100)
print(" ")
```

Output:

```
Missing Value Check:

Person_ID Missing values:  0
Age Missing values:  0
Gender Missing values:  0
Occupation Missing values:  0
Daily_Screen_Time Missing values:  0
Social_Media_Usage Missing values:  0
Night_Usage Missing values:  0
Sleep_Hours Missing values:  0
Stress_Level Missing values:  0
Work_Study_Hours Missing values:  0
Physical_Activity Missing values:  0
Social_Interaction_Score Missing values:  0
Caffeine_Intake Missing values:  0
Smoking Missing values:  0
Alcohol Missing values:  0
Depression Missing values:  0
Anxiety Missing values:  0
Burnout Missing values:  0
```

Inconsistencies Check

To address inconsistencies, I focused on columns containing string/text fields to streamline potential user errors. For example, in a dataset, user input errors are common, such as misspellings or variations in spelling with _ or spaces. As an analyst, one is supposed to normalize values to the most correct version to avoid skewing a dataset. Gender, Occupation, and Physical Activity were the columns I chose, and I used the Python .unique function to identify possible inconsistent values. After running the code, the dataset quality control passed, since the values were already normalized.

Code:

```
print("Person_ID Unique values: ", Person_ID_Unique_values, "\n")
print("Age Unique values: ", Age_Unique_values, "\n")
print("Gender Unique values: ", Gender_Unique_values, "\n")
print("Occupation Unique values: ", Occupation_Unique_values, "\n")
print("Daily_Screen_Time Unique values: ", Daily_Screen_Time_Unique_values, "\n")
print("Social_Media_Usage Unique values: ", Social_Media_Usage_Unique_values, "\n")
print("Night_Usage Unique values: ", Night_Usage_Unique_values, "\n")
print("Sleep_Hours Unique values: ", Sleep_Hours_Unique_values, "\n")
print("Stress_Level Unique values: ", Stress_Level_Unique_values, "\n")
print("Work_Study_Hours Unique values: ", Work_Study_Hours_Unique_values, "\n")
print("Physical_Activity Unique values: ", Physical_Activity_Unique_values, "\n")
print("Social_Interaction_Score Unique values: ", Social_Interaction_Score_Unique_values, "\n")
print("Caffeine_Intake Unique values: ", Caffeine_Intake_Unique_values, "\n")
print("Smoking Unique values: ", Smoking_Unique_values, "\n")
print("Alcohol Unique values: ", Alcohol_Unique_values, "\n")
print("Depression Unique values: ", Depression_Unique_values, "\n")
print("Anxiety Unique values: ", Anxiety_Unique_values, "\n")
print("Burnout Unique values: ", Burnout_Unique_values, "\n")
```

Output:

```
Gender Unique values:  <StringArray>
['Male', 'Female']
Length: 2, dtype: str

Occupation Unique values:  <StringArray>
['Student', 'Employed', 'Unemployed']
Length: 3, dtype: str

Physical_Activity Unique values:  <StringArray>
['Low', 'High', 'Medium']
Length: 3, dtype: str
```

Formatting Error Check

To properly run analysis on a dataset, the analyst must avoid formatting mistakes that can cause columns with data to produce inconsistent results. Special characters such

as @ or $, among others, can cause Python to classify an entire column as an object type rather than a numerical one, thereby preventing the use of values for statistical analysis. I ran the .info() method in Python to verify each data type per column. The dataset passed the QA check, as each numerical value was properly listed alongside the string objects.

Code:

```
df.info()
```

Output:

```
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 18 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Person_ID                 2000 non-null   int64
 1   Age                       2000 non-null   int64
 2   Gender                    2000 non-null   str
 3   Occupation                2000 non-null   str
 4   Daily_Screen_Time         2000 non-null   float64
 5   Social_Media_Usage        2000 non-null   float64
 6   Night_Usage               2000 non-null   int64
 7   Sleep_Hours               2000 non-null   float64
 8   Stress_Level              2000 non-null   int64
 9   Work_Study_Hours          2000 non-null   float64
 10  Physical_Activity         2000 non-null   str
 11  Social_Interaction_Score  2000 non-null   int64
 12  Caffeine_Intake           2000 non-null   int64
 13  Smoking                   2000 non-null   int64
 14  Alcohol                   2000 non-null   int64
 15  Depression                2000 non-null   int64
 16  Anxiety                   2000 non-null   int64
 17  Burnout                   2000 non-null   int64
dtypes: float64(4), int64(11), str(3)
memory usage: 281.4 KB
```

Outlier Check

I checked for outliers using the IQR method on all the numerical variables like Sleep_Hours, Daily_Screen_Time, and Stress_Level. After calculating the bounds, the results showed 0 outliers for every single column. Since all the data falls within the 1.5x IQR range and the min/max values actually make sense for a mental health survey, I didn't have to drop any rows. The dataset passed the outlier check with 100% of the data kept for the project.

Code:

```python
df.info()
print(df.describe(include='all'))

print("_" * 100)
print(" ")
print("Outlier Check: \n")
num_cols = df.select_dtypes(include=['float64', 'int64']).columns

for col in num_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
    print(f"{col} Outliers: {len(outliers)}")
```

Output:

```
Outlier Check:

Person_ID Outliers: 0
Age Outliers: 0
Daily_Screen_Time Outliers: 0
Social_Media_Usage Outliers: 0
Night_Usage Outliers: 0
Sleep_Hours Outliers: 0
Stress_Level Outliers: 0
Work_Study_Hours Outliers: 0
Social_Interaction_Score Outliers: 0
Caffeine_Intake Outliers: 0
Smoking Outliers: 0
Alcohol Outliers: 0
Depression Outliers: 0
Anxiety Outliers: 0
Burnout Outliers: 0

Process finished with exit code 0
```

Conclusion:

After finishing the QA process, the dataset is fully cleared for analysis. I verified that there are zero duplicates in the Person_ID column and no missing values at all, so I didn't have to worry about dropping rows or filling in gaps. The data types are all correct and the string values were already normalized, which means there aren't any weird formatting issues or typos that would mess up the results later. Plus, since the IQR check showed zero outliers, I can move forward with 100% of the data kept. Overall, the dataset is clean and ready to go for the visualization phase.